JavaScript講座【レベル1:入門編】
～ゼロから始めるJavaScriptの基本と仕組み～

2026/03/10

Kazuma Sekiguchi

前回のAgenda

- プログラミングとは何か（入力→処理→出力、状態と副作用）
- JavaScriptの特徴（ブラウザ、動的型付け、イベント駆動）
- JSの記述方法（script、defer、Consoleで試す）
- DevToolsの基本（Console/Sources、エラーの読み方：行番号・スタック）
- 変数・数値・文字列、テンプレート文字列、演算子、比較

今回のAgenda

- if / else、truthy/falsy、論理演算子
- for / for…of / while
- 配列：作成・参照・length・push/pop
- “反復”の考え方

条件（if文）

- 条件に応じて、処理を分けるケースは多い
  - いわゆる判断をさせる場合に利用
  - 条件式を記述して判断させる
  - ifのみ必須
    - 他は任意
  - else ifは複数回利用可能

```
if(条件式1){
        条件式1に当てはまるならここが動作
}
else if(条件式２){
        条件式2に当てはまるならここが動作
}
else{
        条件式に当てはまらない場合に動作
}
```

条件式

- 条件式は不等号などを組み合わせて示す
  - 論理演算子とも呼ぶ
  - その式を満たせば、trueとなり、満たさなければfalseとなる
- if文の条件式に入れることでif文を満たすか満たさないかを示すことができる
  - if(条件式)　の場合、条件式はtrueまたはfalseになることが期待される
  - trueになるような値またはfalseになるような値でも問題は無い

条件式

- 分岐する条件を不等号などの記号を使い表したもの
  - 通常「真か偽か」という結果として示す

```
let a = 4;
if( a > 3){
  //何らかの処理
}
```

```
let a = 4;
if( a >= 4){
  //何らかの処理
}
```

```
let a = 4;
if( a == 4){
  //何らかの処理
}
```

これらは条件式が真になる

```
let a = 4;
if( a < 4){
  //処理されない
}
```

```
let a = 4;
if( a <= 3){
  //処理されない
}
```

```
let a = 4;
if( a == 5){
  //処理されない
}
```

これらは条件式が偽になる

条件式で使う不等号

a > 3

- aが3よりも大きい

a >= 3

- aが3以上である

a < 3

- aが3未満である

a <= 3

- aが3以下である

a === 3

- aが3である

a === ‘js’

- aが文字列のjsである

a !== 3

- aが3では無い

a !== ‘js’

- aが文字列のjsではない

- 文字列は一致（===）か一致していない（!==）かしか使えない
  - 一致は==、不一致は!=でもOKではあるが推奨しない

条件式

- 条件式が成立する場合（真）、該当箇所の処理が行われる
  - 条件式が成立しない場合（偽）、該当箇所の処理は飛ばされる
- 必ずif文から初まり、else文は条件式を付けることができない
  - elseは条件が全て「真」にならない場合に処理される

else ifが無い例

```
let a = 4;
if(a > 5){
  //処理されない
}
else{
  //処理される
}
```

elseが無い例

```
let a = 4;
if(a > 5){
  //処理されない
}
else if(a > 4){
  //処理されない
}
```

else ifとelseが無い例

```
let a = 4;
if(a > 5){
  //処理されない
}
```

条件式

- 条件式にはJSのメソッドを入れることも可能
  - 単一の値を入れることも可能
  - 条件式に記述したものが先に処理される
  - 返り値によって、条件式が真になったり偽になったりする
- 0以外の数、true ，何らかの文字列は全て条件式で真になる
  - 0,false,null,undefined,NaN,空文字だと条件式で偽になる
    - falsy
- 条件式の先頭に「!」を付けると符号逆転する
  - trueならばfalseになり、falseであればtrueになる

```
let isUser = true;
if(!isUser){ //isUserを逆転するので不成立になる
```

条件式

- 条件式は複数組み合わせることも可能
  - AND（&&）　，OR（||）を記述する
  - 条件式はそれぞれ括弧でくくっても良いし、括らなくても良い
  - 括った場合は先に実行されるため、メソッドを使うときなどには括弧で括った方が良い

```
let a = 4;
if((a === 5) || (a > 3)){
  //処理が実行される
}
```

aが5である、またはaが3よりも上である

```
let a = 4;
if((a !== 5) && (a > 3)){
  //処理が実行される
}
```

aが5ではない、かつaが3よりも上である

配列

- 1つの変数に複数の値を格納することのできる変数
- 例）100人の名簿を管理する
  - 変数だと100個作らないと管理できない
  - 変数作成するだけで疲れる
  - 人数が確定していないときは・・・？
    - 変数の数が分からない

```
let name1 = ‘Tanaka’;
let name2 = ‘Suzuki’;
let name3 = ‘Watanabe’;
        :
        :
let name100 = ‘Todoroki’;
```

配列

- 1つの変数に複数の値を入れられれば、変数は1つで済む
  - 変数に格納されている値は順番（インデックス）で管理する
  - 順番は0番から始まることに注意
  - []で全体を括り、入れたいものをカンマで区切って記述
  - 取り出すときは、変数名[順番]で取り出せる
    - 順番で指定したときに値が格納されていないとエラーになる

```
let arr = [‘Tanaka’,’Suzuki’,’Watanabe’,・・・・,’Todoroki’];
let arrNum = [92,47,86,76・・・,87];
let a = arr[0];//Tanakaが格納される
```

配列

- データを取り出すときはインデックスを付けて取り出す

```
let arr = [‘Tanaka’,’Suzuki’,’Watanabe’,・・・・,’Todoroki’];
console.log(arr[0]);//Tanakaが取り出される
```

- インデックスを指定して上書きすることが可能

```
let arr = [‘Tanaka’,’Suzuki’,’Watanabe’,・・・・,’Todoroki’];
arr[0] = ‘Kimura’;
console.log(arr[0]);//Kimuraが取り出される
```

- 配列の後に.lengthを付けることで、要素数を取得できる

```
let arr = [‘Tanaka’,’Suzuki’,’Watanabe’,’Todoroki’];
console.log(arr.length);//4が表示される
```

配列

- 配列の末尾にデータを格納する場合は、配列.push()を利用する

```
let arr = ['Tanaka','Suzuki','Watanabe','Todoroki'];
arr.push('Sato');
console.log(arr[4]);//Satoが表示される
```

- 配列の末尾からデータを取得する場合は、配列.pop()を利用する

```
let arr = ['Tanaka','Suzuki','Watanabe','Todoroki'];
let data = arr.pop();
console.log(data);//Todorokiが表示される
console.log(arr);// ['Tanaka','Suzuki','Watanabe'];となる（Todorokiが削除される）
```

ループ（繰り返し）

- 同じことを何度も記述するのは大変
  - 1回記述して繰り返して貰う方が楽
  - PCの場合、同じことを繰り返すのは極めて高速
- 配列との相性が非常に良い
  - 配列と組み合わせて使うのが一般的

```
for(let i = 1; i <= 100; i++){
  goukei += i;
}
```

1から100まで足し算する

+= で左辺の変数に右辺の値を加えるという意味になる
goukei = i + goukeiでも同じ意味

ループ（繰り返し）

```
for(let i = 0; i < 10; i++){

}
```

初期値

条件式

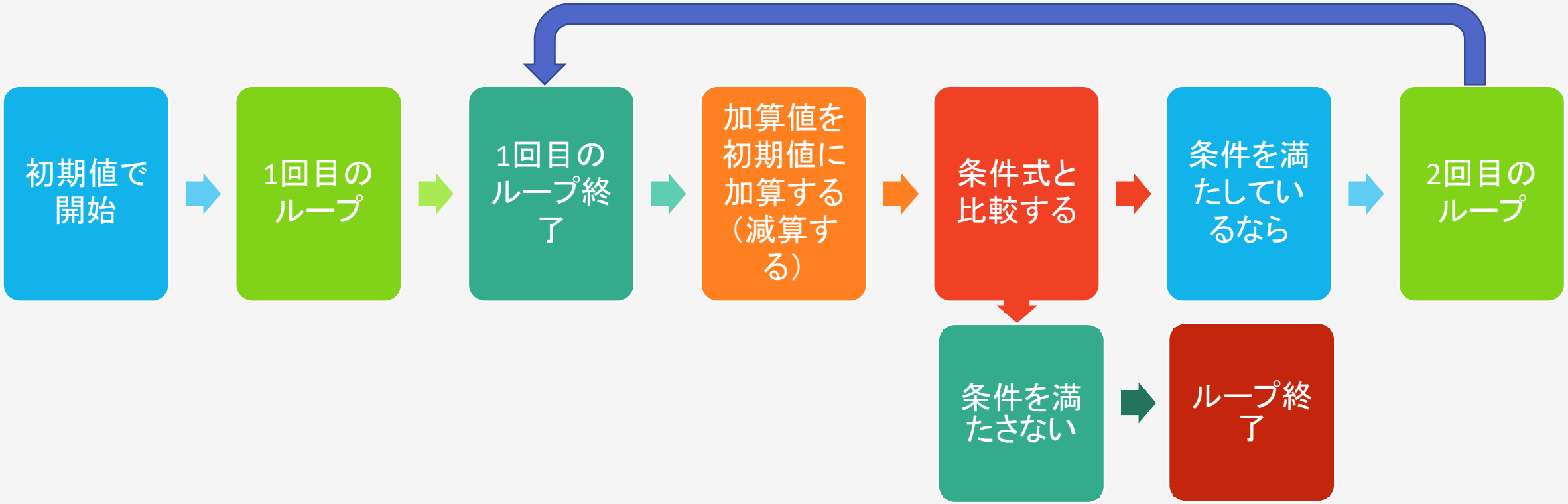
加減値

i++はi = i + 1の略
i--は i = i - 1の略

- 初期値、条件式、加減値の3つを記述する
  - 初期値はletで変数を指定し、初期値を設定する
  - 条件式はループを繰り返す条件を記述する（if文の条件式と同じ）
    - 初期値として指定した変数の値が条件を満たしている間繰り返すようにするのが一般的
  - 加減値で1回のループでどれだけ数を増やすか減らすか指定する

ループ（繰り返し）

- 初期値で設定した値でループを開始し、ループの終わりに来たら加減値を値に加算または減算する
  - 加算または減算した値と条件式を比べて条件を満たしているのならまたループを行う



ループ（繰り返し）

```
for(let i = 0; i < 10; i++){
  //繰り返す内容を記述
}
```

10回繰り返される

複数のfor文を
組み合わせることも可能

```
for(let i = 0; i < 10; i++){
  for(let j = 0; j < 10; j++){
  //繰り返す内容を記述
  }
}
```

100回繰り返される

- for文の初期値はlet i ＝ 0;が多い（またはi ＝ 1;）
  - iはiterator（反復子）の略として使われる
  - 変数iとかjはfor文の初期変数という認識
  - 別の変数を使ってももちろん可能

ループと配列

- 配列は0からの番号で管理される値の集合体
- 配列の中の値は基本的に一気に扱うことができない
  - 1つずつ取り出して処理をする必要がある
  - 繰り返す必要がある

- for文は0からの数字で回数を管理できる

- 組み合わせるととても都合が良い

ループと配列

```
let fruitlist = ‘apple’,’orange’,’grape’,’strawberry’];
for(let i = 0; i < fruitlist.length; i++){
  document.writeln(fruitlist[i]);
}
```

4つの果物名で配列を作成

配列（Arrayオブジェクト）はlengthプロパティで配列の個数を取得できる

インデックスを指定すれば配列の該当箇所の値を取得できる
このときのiはループの回数を数えているiのこと→勝手に増えていく値なので、全部処理ができる

- 配列のlengthプロパティで配列の数を数えることが可能
  - それを条件式に入れることでループの終わりとする
- 配列はインデックスでアクセスができるので、[i]とすれば、勝手にiの値が増えていくことで、配列の違う値を扱うことが可能

for..of文

- 順番を管理する変数i（添字とも呼ぶ）が不要ながら、ループさせることができる構文

```
let arr = [‘Tanaka’,’Suzuki’,’Watanabe’,’Todoroki’];
for(let data of arr){
  console.log(data);//Tanaka,Suzuki,Watanabe,Todorokiが順番に取り出される
}
```

- 余分な変数を使わずに配列内のデータを取得することが可能
  - 配列を扱うときは楽

ループを途中で飛ばす、止める

- ループで処理している途中で別条件を満たした段階でループを終了させたり、途中で次の値の処理に移るようにすることが可能
  - for文、for..of文、while文で使える

```
let sum = 0;
for(let i = 0; i < 10; i++){
  sum += i;//sum変数にiの値を加算する
  if(sum > 20){//変数の値が20を超えたら
    break;//ループを抜け出す
  }
}
```

breakを通った場合は、ループ処理が終了する（ループが複数の場合は、直接のループだけ終了）

```
let arr = [70,65,81,46];
for(let i = 0; i < arr.length; i++){
  if(arr[i] > 60){//60よりも大きければ
    continue;//次の値を処理する
  }
  console.log('不合格');
}
```

continueを通った場合は、ここは処理されない（ループが複数の場合は、直接のループだけ飛ばされる）

while文

- ループをする点では他と同じ

```
let i = 0;
while(i < 10){//括弧内の条件を満たす限り繰り返す
    console.log(i);
    i++;//変数iに1を加算する
}
```

- while（条件式）で条件を満たしている限り繰り返す
  - 記述を間違えると永久にループする点に注意が必要

反復の考え方

- 同じ処理を対象の数だけ自動で繰り返すのが反復
  - コードを増やすのでは無く、繰り返し条件を記述してループ処理にする
- 配列を上手く利用することで、数が不特定なものをコードで扱うことができる
  - 1つずつ変数に格納していたら、数が不特定なものは扱えない
- 対象と繰り返す処理（毎回やること）、終了条件を考えて言語化できると、ループ処理に起こすことができる

ありがとうございました。

また次回。